// performance

Django × PostgreSQL

making your ORM behave in production

A practical, example-driven tour of smart field selection, N+1s, index strategies, subquery pitfalls, query rewrites, and reading EXPLAIN ANALYZE without panic — all pulled from real incidents, not toy snippets.

Oleksandr // 2026

What we'll cover

Six stops, each anchored in something that broke in production.

01

Mental model

The pipeline. ORM. Query planner.

02

Debugging queries

Print SQL, run EXPLAIN ANALYZE

03

Reading EXPLAIN

Scan types, estimates vs actual, panic control

04

Why queries are slow

N+1, stats, indexes, table size

05

Index strategies

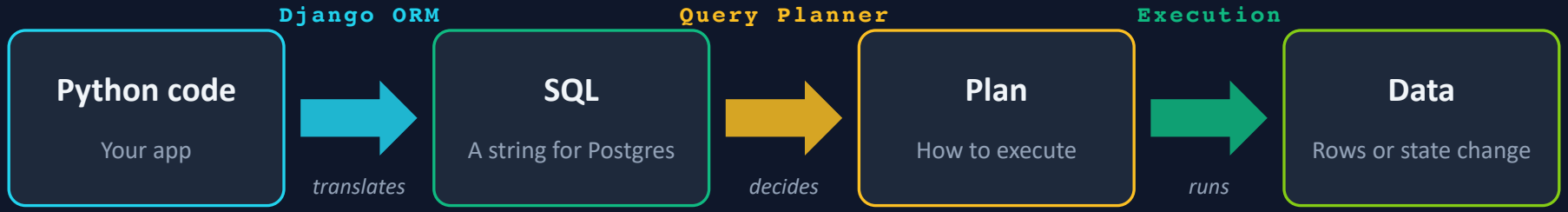
B-tree, GIN, partial, functional

06

Bad patterns & scale

Bloat, churn, partitioning, replicas

How a Django query actually flows



When a query feels slow, name the stage first — then you know whose rules apply.

Django ORM is just Active Record

A Python mirror of your database. Three mappings, nothing magic.

`table`	→	`model`	A class per table
`column`	→	`field`	A class attribute per column
`row`	→	`model instance`	A Python object per row

Three rules to internalise

These are the levers behind almost every Django performance fix.

Fields are eager

When you fetch a row, all of its columns come with it by default. Every byte costs time.

Relations are lazy

Foreign keys and reverse accessors don't load until you touch them — exactly where N+1s hide.

QuerySets are lazy

Chaining filter / exclude / order_by builds SQL in Python. Nothing hits the DB until you iterate, slice, or len().

Read that as: a QuerySet is a recipe, not a result.

QuerySets are lazy — three quick puzzles

Read each snippet. Guess the query count before you peek at the answer.

A Build

```
qs = User.objects.filter(
  active=True
)
```

How many queries?

?

B Evaluate

```
if qs:
    print(qs[:2])
```

How many queries?

?

C Subquery

```
qs1 = User.objects.filter(
  active=True
).values_list(
  "id",
  flat=True
)
qs2 = Profile.objects.filter(
  user__in=qs1
)
for p in qs2: …
```

How many queries?

?

Meet the query planner

It enumerates plans, estimates each one's cost, and runs the cheapest.

1. Parse & rewrite

Turn your SQL into a tree. Expand views and rules.

2. Plan & cost

Enumerate scan and join strategies. Cost each using table stats.

3. Execute

Run the cheapest plan. Cache it (sometimes) for reuse.

You don't drive it — you influence it. Tools: indexes, LIMIT / ORDER BY, column selection, table stats.

Step 1: see the SQL Django actually runs

You cannot tune what you cannot read. Turn the lights on first.

shell_plus

```
manage.py shell_plus --print-sql
```

Interactive shell that prints the SQL behind every QuerySet you evaluate. The fastest way to poke at queries by hand.

Print a QuerySet

```
print(qs.query)
```

Works without shell_plus. Shows the exact SQL the ORM would generate — useful for code review and tests.

DEBUG logging

```
LOGGING: django.db.backends → DEBUG
```

Every query your app runs ends up in the log, with timing. Indispensable for hunting N+1s in real request flows.

Step 2: ask the database what it's doing

EXPLAIN is the plan. EXPLAIN ANALYZE actually runs it — with real numbers.

EXPLAIN

The plan the planner intends to use.

- Fast, no side effects
- Cost estimates only
- Use for exploratory reads

EXPLAIN ANALYZE

The plan plus real execution numbers.

- Actual rows, timings, buffers
- Use a read replica for big tables
- Wrap writes in a rolled-back tx

`Get to a psql prompt fast: manage.py dbshell`

Worked example: from logs to plan

The whole debugging workflow on a single one-row lookup.

1. settings.py — enable SQL logging

```
LOGGING = {
    "version": 1, "disable_existing_loggers": False,
    "handlers": {"console": {"level": "DEBUG", "class": "logging.StreamHandler"}},
    "loggers":  {"django.db.backends": {"level": "DEBUG", "handlers": ["console"]}},
}
```

2. shell_plus — every QuerySet logs its SQL

```
>>> User.objects.get(id=1)
(0.002) SELECT "auth_user".* FROM "auth_user" WHERE "auth_user"."id" = 1 LIMIT 21;
<User: admin>
```

3. dbshell — EXPLAIN ANALYZE gives plan + actual timings

```
Limit  (cost=0.14..8.16 rows=1)  (actual time=0.036..0.038 rows=1 loops=1)
  ->  Index Scan using auth_user_pkey   Index Cond: (id = 1)     Planning: 0.150 ms  Execution: 0.455 ms
```

EXPLAIN ANALYZE without the panic

Four questions that answer 90% of plans.

1 What kind of scan is on the bottom?

Seq Scan on a hot table = an index is missing or can't be used. Index Scan / Bitmap Scan = planner found your index.

2 Do estimates match actual rows?

If rows=1 and actual=50,000, your stats are stale or a filter is misleading the planner. Run ANALYZE.

3 Index Cond vs. Filter?

Index Cond: condition used the index (fast). Filter: condition applied after reading rows (slower).

4 Where is the time actually spent?

Follow the largest 'actual time' — often a sort, a nested loop, or a scan feeding a join. That's your target.

Five root causes. Everything else is a symptom.

Learn to name the cause before picking the fix.

Unnecessary work

Queries and sorts you don't actually need in this code path.

N+1 queries

Missing prefetch_related / select_related on relations.

Missing indexes

The planner has no choice but to scan the whole table.

Size amplifies everything

A linear scan on 10k rows is fine. On 10M it isn't.

Stale / skewed statistics

Calculated on 100 values by default — the planner often sees only a partial picture.

The N+1 query problem

One query to list, one per row for related data — the classic Django slowdown.

The smell

- A simple endpoint returns in ~2s
- Logs show 100+ nearly identical queries
- Each one differs only by a single id
- Table scans are not the bottleneck — round-trips are

The fix

- select_related(...) for FK / OneToOne — JOIN in one query
- prefetch_related(...) for M2M / reverse — second IN query
- Prefetch(queryset=...) to control what gets loaded
- Verify with DEBUG logging — query count should drop to O(1)

Rule of thumb: if a serializer loops over children, a prefetch is missing.

Fetch only the fields you actually use

Every column shipped over the wire is latency you didn't need.

`only()`

Load only these columns; defer the rest. Great for list endpoints that render 5 of 40 fields.

`defer()`

The inverse — load everything except these. Useful for skipping big TEXT / JSON blobs.

`values()` / `values_list()`

Skip model instantiation entirely when you just need raw data for a response or a set.

`update_fields=[...]`

On save(), touch only the columns that changed. Cuts write amplification and lock scope.

Drop sorts and DISTINCTs you don't need

The cheapest optimisation is the work you stop doing.

A model's default ordering is still an ORDER BY

Meta.ordering applies even to an Exists check. Call order_by() to wipe it when you don't care.

DISTINCT usually forces a sort

It's often a band-aid over a bad JOIN. Fix the JOIN (or use EXISTS) and you can drop the DISTINCT too.

Disk sorts are an order of magnitude slower

When the sort spills past work_mem you start reading and writing temp files. EXPLAIN ANALYZE will say 'external merge'.

04 / FIX #4

Prefer EXISTS over JOINs when you only need a yes/no

Symptom

Filter using a reverse relation builds a JOIN — sometimes several. Planner decides row counts are unpredictable and picks a bad shape.

Why it hurts

JOINs duplicate rows, force DISTINCT, and wedge the planner into sort-merge or hash plans. Expensive on wide tables.

The fix

Rewrite as Exists(sub). Postgres short-circuits on the first matching row and the outer query stays narrow.

Also worth trying: materialise ids with a subquery, then filter the outer query by id__in.

Split extensive OR queries with UNION

The planner hates big OR. Help it by feeding smaller, well-shaped pieces.

Why one big OR struggles

Each branch of the OR may favour a different index. The planner often gives up and scans the whole table — and OR rarely eliminates a sort.

Refactor as UNION

- Each branch becomes a separate QuerySet with its own indexed filter
- qs1.union(qs2, qs3) — each piece gets its own plan
- Deduplication is O(n log n), but n is smaller than a full scan

04 / FIX #6

Plan cache: great idea, sharp edges

Reusing plans saves CPU. It also lies to you in multi-tenant systems.

What it does

After a few runs of the same prepared statement, Postgres freezes a generic plan and reuses it, skipping planning cost.

Why multi-tenant apps suffer

A plan optimal for a 100-row tenant may be disastrous for a 10M-row one. Same query, same app, wildly different profiles.

Practical knobs

Discard cached plans (DEALLOCATE), pass parameters that nudge a re-plan, or make the query shape tenant-aware.

Gotcha

Some functional indexes cannot be used in cached (generic) plans — the planner needs the concrete parameter to prove a match.

B-tree and GIN: the two workhorses

Start with B-tree. Reach for GIN when the column is a collection.

B-tree

The default. Fast lookups, range scans, and ORDER BY.

- = and range filters
- ORDER BY without a sort step
- Leftmost-prefix rule applies to compound keys
- Not great for contains / fuzzy / JSON

GIN

Generalised inverted index. Built for "does this contain that?".

- Arrays — a lighter alternative to M2M tables
- Full-text search on tsvector
- Trigram (pg_trgm) for icontains / fuzzy match
- btree_gin — combine equality + contains in one index

Partial & compound indexes

Two techniques that quietly delete entire classes of slow query.

Partial / sparse

An index that only covers the rows you query — with a WHERE clause.

- Smaller → faster to scan and update
- Perfect for "status = 'active'"
- Queryset must match the WHERE exactly

Compound (multi-column)

One index on (a, b, c). The order matters.

- Queries must use the left-most column
- Great for (tenant_id, status, created_at)
- Avoid stacking single-column indexes you never combine

Functional, sort-optimised, and covering indexes

When the default B-tree isn't quite enough, these three get reached for next.

Functional

Index an expression — LOWER(email), EXTRACT(dow FROM created_at) = 0 for weekend rows. Only used if the query uses the same expression.

Sort-optimised

Match ORDER BY column order and direction. Eliminates the sort step entirely, even on large result sets.

Include / covering

INCLUDE (col1, col2) ships extra columns inside the index. The planner can answer the query without touching the table.

When to use them

Rare or dynamic queries, complex ORs, or write-heavy tables where you want fewer but smarter indexes.

Functional indexes can be excluded from cached / generic plans — test them with real parameters.

Four indexes, one query — each tighter

Same data, same SELECT. The plan sharpens as the index learns the query.

```
SELECT id, created FROM events WHERE tenant_id=1 AND created > '2025-12-31' AND NOT archived
ORDER BY created;
```

1 **Sequential scan** — *Postgres reads every row. Fine at 10k rows, a disaster at 10M.*

— no index —

2 **Index scan** — *Selective on date ranges, sort-optimised — and all selected fields already live inside the index.*

```
CREATE INDEX ON events (created);
```

3 **Compound index** — *Narrows by tenant before the date range. Leftmost-prefix still serves tenant-only queries.*

```
CREATE INDEX ON events (tenant_id, created);
```

4 **Partial / sparse** — *Only live rows indexed → smaller, faster to scan, cheaper to maintain.*

```
CREATE INDEX ON events (tenant_id, created) WHERE NOT archived;
```

When the query grows: covering and functional

Add a SELECT column or push a predicate into the index — Postgres keeps up if you shape the index right.

5

Covering index — SELECT gained a new column

```
SELECT id, created, name FROM events ...
```

```
CREATE INDEX ON events (tenant_id, created, name) WHERE NOT archived;
```

```
CREATE INDEX ON events (tenant_id, created) INCLUDE (name) WHERE NOT archived;
```

INCLUDE keeps name out of the B-tree key → same lookup, cheaper writes.

6

Functional index — predicate gained an expression

```
... WHERE EXTRACT(DOW FROM created) IN (0, 6) ...
```

```
CREATE INDEX ON events (tenant_id, created)
  WHERE EXTRACT(DOW FROM created) IN (0, 6) AND NOT archived;
```

Push the expression into the index → weekend rows only, no scan-and-filter.

Update-heavy tables rot quietly

Every UPDATE leaves a dead tuple. Over time, bloat makes everything slow.

Autovacuum

Reclaims dead row space in the background. Tune per-table when it can't keep up.

Index bloat

Indexes grow faster than tables for UPDATE-heavy rows. Watch pg_stat_user_indexes.

VACUUM FULL

Rewrites the table. Takes an exclusive lock. Avoid during traffic hours.

pg_repack

Same outcome, minimal locking. The default tool for production reorgs.

Delete-then-create churn: mind the ID sequence

A year later you're out of INT32 and facing a scary migration.

The pattern

Jobs, syncs, or imports that DELETE the previous batch and INSERT a new one. Rows churn, but the id sequence only goes up.

Prevention & mitigation

- Start with BigAutoField / BigIntegerField on anything you might churn
- Prefer UPDATE-in-place over delete / re-insert when the row identity is stable
- Migrating INT → BIGINT later requires locking or a shadow column rewrite

Two more tools to know about

Not always needed, but worth recognising when the time comes.

Table partitioning

Split a huge table into smaller physical pieces by date, tenant, or hash. The planner prunes partitions it doesn't need.

Caveat: Django doesn't natively support FKs to the partition key. Plan the data model around that.

Primary / read-replica split

Send heavy reads and reports to a replica, keep writes on the primary. Scales read throughput without touching application logic much.

Caveat: replicas lag. Don't read-your-own-write on them — route critical reads to primary.

What to carry back to your codebase on Friday

A short checklist, ordered by return on effort.

- **Log every query**

  Turn DEBUG on in dev, watch the count in tests.

- **Kill N+1s first**

  select_related, prefetch_related. Single biggest wins.

- **Ship only used fields**

  only(), update_fields, values().

- **Prefer EXISTS to JOIN**

  Only a boolean → use EXISTS. Need the rows → use prefetch.

- **Index the queries you run**

  Compound order matters. Partial shrinks. Functional unlocks.

- **Befriend EXPLAIN ANALYZE**

  Four questions get you most of the way.

// questions?

Thank you.

Go measure something slow today.

Next time a query feels slow: print the SQL, run EXPLAIN ANALYZE, name the root cause, then pick the fix.